

```

PORTS      NAMES
003d069f511b  mysql_local "docker-entrypoint.sh" 2 minutes
ago        Up 2 minutes      0.0.0.0:3306->3306/tcp
hopeful_torvalds

```

```

$ mysql -h127.0.0.1 -u root -proot
Type 'help;' or '\h' for help. Type '\c' to clear the current
input statement.

```

```

mysql>
mysql> exit

```

That's it! You have created an image for MySQL and started a container running MySQL server on port 3306. All we needed were two lines of code in our Dockerfile, which can now be used across operating systems with the same commands to achieve the same output. There are a few more commands that we need to know before we move forward.

 **Note:** If you get an error while running the container, it must be because you already have MySQL running locally. Stop your local MySQL by using `sudo service mysql stop`.

```

<code: shell>
# Stopping a container. The 'id' is taken from 'docker ps'
$ docker stop 003d069f511b
# List all containers (running and stopped)
$ docker ps -a
# Delete a container
$ docker rm <id>
#Delete an image
$ docker rmi IMAGE
#Delete ALL containers. (Use with caution)
$ docker rm -f $(docker ps -a -q)
#Delete ALL images (Use with caution. It deletes every image
on you machine!)
$ docker rmi -f $(docker images -q)

```

You can play around with this set-up as much as you wish. Try modifying the Dockerfile to build new images with different configurations. Consider achieving this with what you have just learned.

1. Run two MySQL containers on different ports (3306 and 3307).
2. Create an image which allows a blank root password.
3. Create an image with a database named 'Books' and a user with the name 'Librarian'.

Moving on to our Rails application, let's create a Dockerfile and `docker-compose.yml` inside the application directory. Just like MySQL, you can also get base images for Ruby, and we will use 'Ruby:2.3.1' in our Dockerfile to build the image for the app. The `yaml` file is the template for our environment, and it will be used to spin off dependent services in separate containers, saving us from the overhead

of linking them. 'Compose' is a tool for defining and running multi-container Docker applications. With Compose, you create and start all the services from your configuration using a single command.

```

<code: shell>
$ cd ~/dock/rails-mrd-demo
$ touch Dockerfile docker-compose.yml

<code Dockerfile>
#Using ruby 2.3.1 as our base image
FROM ruby:2.3.1
#
# Installing dependencies. You can execute any linux
command using RUN
#
RUN apt-get update -qq && apt-get install -y build-
essential libpq-dev nodejs
#
#Prepare directory where the application will be copied
inside the image
RUN mkdir /myapp
#
#Change working directory (cd /myapp)
WORKDIR /myapp
#
# Copy over Gemfile and Gemfile.lock. These files have all the
ruby needs defined.
ADD Gemfile /myapp/Gemfile
ADD Gemfile.lock /myapp/Gemfile.lock
#
# Execute bundle install
RUN bundle install
#
# Copy contents from current directory to /myapp.
ADD . /myapp

```

The Dockerfile syntax is minimal and simple to understand. In this file, we started from a base Ruby image, installed OS packages, copied files, executed commands like `bundle install` and finally copied all our code inside the image. As discussed earlier, we have the option of using 'docker build', but we will use 'docker-compose build' instead. It will read the `yaml` and create three containers for us, which all talk to each other by service names. This means that your `web` container can communicate with the database with `mysql` as an endpoint and the same is true for Redis. Here is the `docker-compose.yml`:

```

<code docker-compose.yml>
version: '2'
services:
  mysql:
    image: mysql

```